Voter API: Multi-Tier Deployment on Kubernetes

Solution Documentation

1. Requirement Understanding

The task was to build a two-tier app on Kubernetes: a microservice in front of a database, containerized and deployed through manifests. I built the service tier as a .NET 10 REST API for managing voter records, running as four pods, and used a single PostgreSQL instance for the data tier.

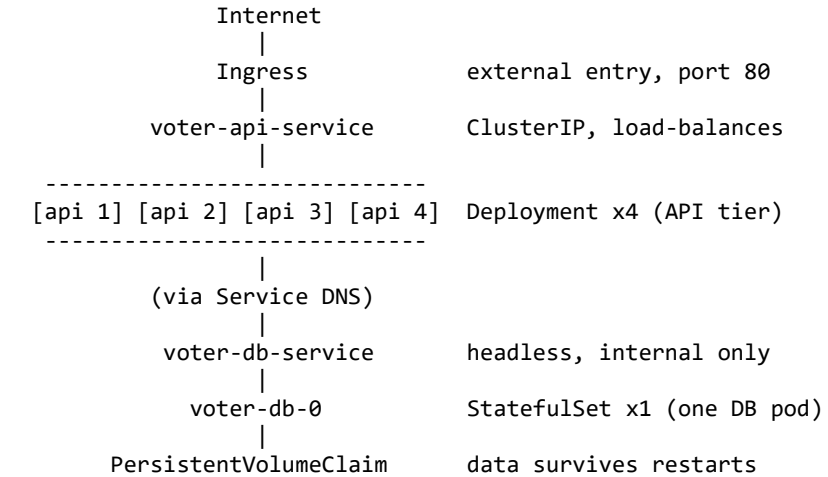
The API has to be reachable from outside the cluster, run several replicas with rolling updates, recover on its own when a pod dies, and scale on CPU. The database runs as a single instance, stays inside the cluster, and must keep its data when its pod is recreated. Configuration comes from a ConfigMap, the database password from a Secret, and the two tiers talk over a Service DNS name rather than pod IPs.

2. Assumptions

- A managed cluster (GKE) is already running, with an NGINX Ingress Controller behind a LoadBalancer that gives the Ingress an external IP.
- A default StorageClass is available to provision the PersistentVolumeClaim.
- The container image is pushed to Docker Hub and the cluster can pull it.
- metrics-server is installed so the autoscaler can read CPU usage.
- One database replica is enough for this workload.

3. Solution Overview

The system is two tiers in one cluster: four API pods behind a single Service and Ingress, and one database pod with its own storage. The diagram below shows the whole picture at a glance.



At runtime a request goes from the client to the NGINX Ingress on the external IP, then to the API Service, which load-balances it to one of the API pods. That pod reaches PostgreSQL through an internal headless Service, never by pod IP. Every response carries X-Pod-Name and X-Node-Name headers, so it is easy to see which pod and node handled the call.

The API is split into a controller layer, a service layer for the voter rules, and an EF Core data layer. The schema and seed data are EF Core migrations that live in the project. During the image build they are compiled into a small efbundle executable, and a Kubernetes Job runs that bundle to apply them. EF keeps track of what it has already applied, so the Job is safe to run on every deploy.

The manifests are applied in an order that follows how the parts depend on each other:

- 1. Create the namespace.
- 2. Apply the connection ConfigMap and the password Secret.
- 3. Create the PVC, the PostgreSQL StatefulSet, and its headless Service, then wait for the database pod to be ready.
- 4. Run the migration Job and wait for it to finish, so the schema and seed exist before any API pod serves traffic.
- 5. Apply the API Deployment (4 pods), its Service, and the Ingress. A pod only joins the load balancer once its readiness probe confirms it can reach the database.
- 6. Apply the Horizontal Pod Autoscaler, which then scales the API between 4 and 10 pods based on CPU.

A deploy script runs these steps in order, so the database and its migration always come before the API. Persistence comes from the StatefulSet mounting a PVC, so deleting the database pod brings it back with the same data. Self-healing comes from the Deployment, which recreates any API pod that is deleted or crashes.

4. Justification for the Resources Utilized

Each piece was chosen for a specific reason:

- .NET 10 and PostgreSQL: a fast, lightweight API runtime and a free, dependable database that suits a single-table CRUD service.
- EF Core migrations with a bundle: schema changes stay with the code and apply the same way every time, with no separate SQL files to keep in sync.
- ConfigMap and Secret: connection details stay out of the image, and the password stays out of plain YAML.
- StatefulSet with a PVC and a headless Service: stable storage that survives restarts, and a database reachable only inside the cluster.
- Deployment with rolling updates, a Service, and an Ingress: zero-downtime updates, self-healing, internal load balancing, and one external entry point.

- HPA with CPU requests and limits: the cluster reserves only what the API needs and adds pods only while load is high.

Saving cost (FinOps). **A few deliberate choices keep the bill close to actual usage**:

- We set the API requests and limits from what the pods really use, read with kubectl top, so each node holds more pods and we run fewer nodes.
- The HPA scales the API on CPU with a 60% target, and we tuned it to drop the extra pods about a minute after load falls, so we are not paying for idle pods between traffic spikes.
- The database stays at one replica on a small, right-sized volume instead of an over-provisioned disk.